

Research on Human Action Recognition Based on Edge Intelligence

Di Wang*

China North Vehicle Research Institute
Fourth Technology Department
Beijing, China

*Corresponding author: wangdi305@163.com

Zhonglin Yang

China North Vehicle Research Institute
Fourth Technology Department
Beijing, China

Gaotian Liu

China North Vehicle Research Institute
Fourth Technology Department
Beijing, China

Abstract—Intelligent recognition algorithms deployed on edge devices offer strong real-time processing capabilities and high security for online video image analysis. However, real-time video image recognition remains challenging, particularly when deploying computationally intensive action recognition models on resource-constrained edge devices. This paper introduces a dual-network human action recognition method that leverages the strengths of both 2D and 3D convolutional neural networks (CNNs). Initially, a 2D CNN detects human targets, followed by the generation of time-sequence data for these targets using a matching method. Human action recognition is then achieved through a 3D CNN. This method addresses the challenges of extracting and modeling temporal action characteristics, enabling accurate human action localization and classification. Five distinct action recognition datasets were constructed to train the 3D CNN, which were subsequently deployed on an edge intelligent platform. Empirical testing of the action recognition system demonstrated a recognition speed of 10 frames per second, with accuracy exceeding 80% for certain action categories.

Keywords—Action Recognition; Deep Learning; Convolutional Neural Network; Edge Intelligence

I. INTRODUCTION

Recognizing human action in video images holds significant engineering and academic value, and has always been a hot topic in the field of computer vision [1], with broad application prospects in public safety, autonomous driving, and human-

computer interaction. In the field of public safety, human action recognition can identify actions that endanger public safety, such as fighting, theft, and crowding, helping staff to improve their response speed to emergencies; in the field of autonomous driving, accurately recognizing the action of pedestrians around vehicles helps autonomous vehicles make correct decisions and enhances driving safety; in the field of human-computer interaction, accurately recognizing users' actional intentions is a prerequisite for intelligence, which helps to improve user experience. In recent years, researchers have conducted extensive research on action recognition, especially using deep learning methods represented by CNN [2] to automatically extract features of human action, achieving certain results, but there is still a significant gap from practical application. This is mainly because in real scenarios, the scene is changing, and human action has a strong temporal nature. Extracting and modeling the temporal action of individuals are two difficulties in action recognition [3].

In response to the problem of extracting temporal information of human action, a dual-network method for human action recognition was designed, as shown in Figure 1. First, a 2D CNN is used to detect human targets, then through a Human Matching Module, temporal information is formed for multiple targets and input into a 3D CNN, ultimately achieving the recognition of human action.

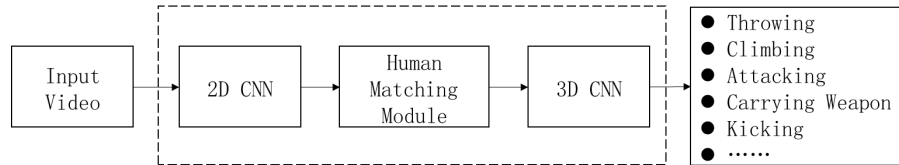


Figure 1. Schematic diagram of the dual-network action recognition method

Addressing the problem of modeling temporal information of human action, a 3D CNN was designed. Intuitively, video images have three dimensions: the width W , height H , and the number of frames L . A 2D convolutional kernel can only move in two directions on the input video, producing a 2D features, as shown in Figure 2(a). In contrast, a 3D convolutional kernel [4] is an extension of the 2D convolutional kernel, adding a temporal dimension to the spatial dimensions, thus expanding

the 2D convolutional kernel into a 3D convolutional kernel. The 3D convolutional kernel can move in three directions, producing a 3D feature map, as shown in Figure 2(b). Therefore, 3D CNNs can simultaneously learn the spatial and temporal features of video images, which is clearly more advantageous [5].

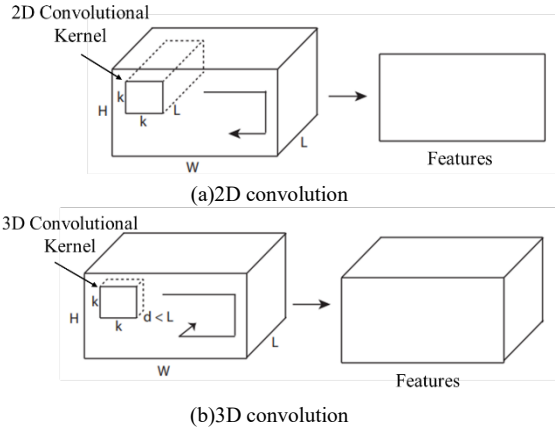


Figure 2. Comparison of 3D convolution and 2D convolution

Ultimately, the aforementioned 2D CNN and 3D CNN were converted and deployed on the edge intelligence platform Atlas200DK, achieving the localization and classification of human action in real-time video images.

II. METHODS

A. 2D CNN

The 2D CNN is primarily used for detecting human targets, which prepares the groundwork for the formation of human temporal data. Designed based on YOLOv8—a leading SOTA model—this 2D CNN is widely used in the field of object detection due to its rapid detection speed and high precision [6]. The model is composed of three main components: the Backbone, Neck, and Head, as depicted in Figure 3. The Backbone utilizes the CSPDarknet architecture, which includes multiple modules such as CBS, C2f, and SPPF, all of which are employed to extract features from the input image. The Neck is comprised of two parts: feature pyramid networks (FPN) and path aggregation network (PAN), which effectively integrate high-level features with low-level features. The Head is made up of three decoupled heads of varying sizes, significantly enhancing detection accuracy by separating the detection and classification tasks. Further-more, the model replaces the Anchor-Based approach with Anchor-Free to more accurately pinpoint the location and category of objects.

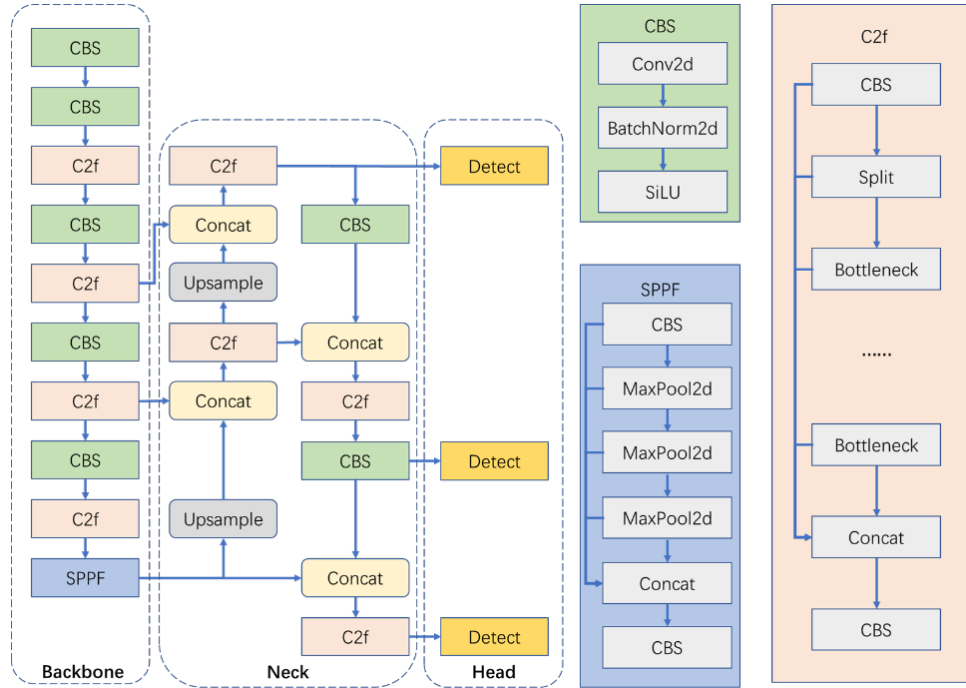


Figure 3. YOLOv8 network architecture

1) Backbone

The Backbone constructed by stacking multiple convolutional layers, pooling layers, and activation layers, serves to extract multi-level features from images. When a 640×640 pixels image is processed through the backbone network, it outputs feature maps of varying sizes, with 20×20 , 40×40 , and 80×80 scale feature maps selected for detecting targets of different sizes. The smallest component of the Backbone, CBS, primarily consists of two-dimensional convolution (Conv2d), batch normalization (BatchNorm), and the SiLU activation function. The Conv2d employs dilated convolution and group convolution techniques. Dilated convolution expands the receptive field, thereby obtaining more

information without increasing computational load. Group convolution reduces the number of network parameters, thereby lessening the burden on the device. Batch normalization primarily addresses the slow training issues caused by uneven data distribution. The SiLU activation function is differentiable across its entire domain, which stabilizes gradient calculations during backpropagation, and, due to its non-monotonic nature, it can suppress excessively large weights, serving a regularization function.

Another component of the Backbone, C2f, includes CBS and Bottleneck structures, mainly drawing on the concept of residual connections. After passing through the CBS, the data is split in

half along the channel dimension, followed by multiple Bottlenecks in series, with the outputs of each Bottleneck being fused at the end. This alleviates the problem of vanishing gradients during training when the network depth increases. The final component of Backbone is spatial pyramid pooling fast (SPPF), an upgraded version of spatial pyramid pooling. By changing the parallel structure of SPP to a serial one, it significantly reduces the amount of parameter computation and enables output of arbitrary sizes. Max-Pool2d refers to two-dimensional maximum pooling.

2) Neck

The Neck is utilized for integrating shallow spatial feature information with deep semantic feature information, thereby enhancing the network's feature representation. The FPN operates in a top-down manner, conveying deep semantic information to shallower layers. Subsequently, the PAN adds a bottom-up pyramid following the FPN, facilitating the transfer of shallow spatial feature information to deeper layers. The interplay between these two structures enhances the network's capability to extract features from targets of varying scales.

3) Head

The Head is employed to generate the final object detection boxes on the feature maps, where the detection task is decomposed into two components: localization (Bbox Loss) and classification (Cls Loss). The localization component encompasses two loss functions, namely the CIoU loss function and the distribution focal (DFL) loss function. Initially, the DFL loss function optimizes the distribution probabilities of the predicted box boundaries and label distributions, followed by the conversion of the distribution probabilities back into predicted bounding boxes. Subsequently, the CIoU loss function calculates the discrepancies between the predicted boxes and the ground truth boxes, thereby continuously refining the location of the predicted boxes.

The DFL loss function employs cross-entropy operations to optimize the probabilities of positions closest to the left and right labels, with the calculation detailed as follows:

$$L_{DFL} = -((y_{i+1} - y) \log S_i + (y - y_i) \log S_{i+1}) \quad (1)$$

y denotes the label value, while y_i and y_{i+1} represent the two adjacent predicted values surrounding the label. Correspondingly, S_i and S_{i+1} denote the predicted probabilities associated with these values, respectively.

The calculation of the CIoU loss function is as follows:

$$L_{CIoU} = 1 - \left(IoU - \frac{\rho^2}{c^2} - \alpha v \right) \quad (2)$$

intersection over union (IoU) represents the ratio of the intersection area to the union area between the predicted bounding box and the ground truth bounding box. The term ρ^2 signifies the square of the distance between the centers of the predicted box and the ground truth box. The term c^2 refers to the square of the diagonal length of the smallest box that can encompass both the predicted box and the ground truth box. The parameter v is a measure of the consistency between the aspect ratios of the two boxes, which is calculated as shown in Equation

(3). The parameter α is a positive weighing factor, the calculation of which is presented in Equation (4).

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w_{gt}}{h_{gt}} - \arctan \frac{w_p}{h_p} \right)^2 \quad (3)$$

$$\alpha = \frac{v}{1 - IoU + v} \quad (4)$$

In this context, of object detection, w_{gt} and h_{gt} represent the width and height of the ground truth bounding box, respectively, while w_p and h_p represent the width and height of the predicted bounding box, respectively. The complete intersection over union (CIoU) loss function incorporates factors such as the overlapping area between the ground truth and predicted bounding boxes, the distance between their centers, and the aspect ratio consistency, thereby enhancing the accuracy of the predicted bounding boxes.

In the classification component, a binary cross-entropy (BCE) loss function is employed to output the confidence level for each class, with the maximum value indicating the confidence of the anchor box. The expression for the BCE loss function is as follows:

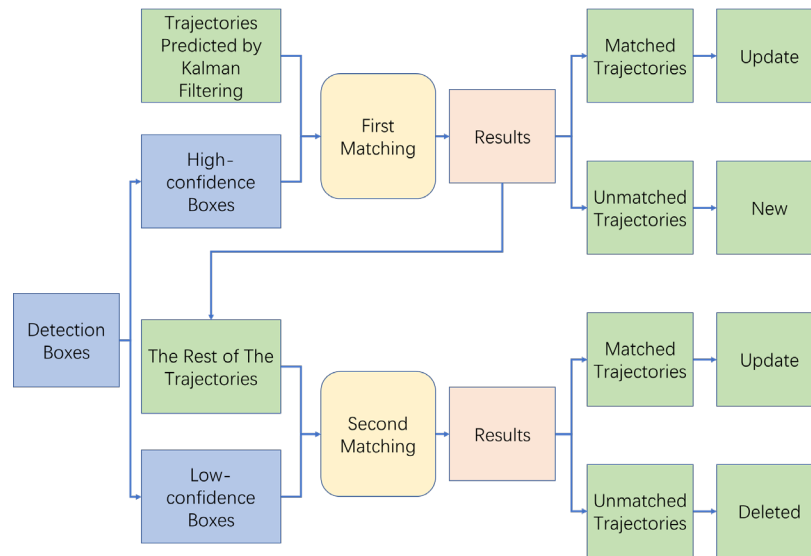
$$L = \frac{1}{N} \sum_i -[y_i \log p_i + (1 - y_i) \log(1 - p_i)] \quad (5)$$

y_i denotes the label of sample i , which is encoded using one-hot encoding. p_i represents the predicted probability of sample i being a positive class, and N signifies the total number of samples.

B. Human Matching Module

The Human Matching Module utilizes the human targets detected by the 2D CNN to deduce their motion trajectories within the video and generates temporal information for each target. The Human Matching Module consists of two critical steps: First, employing the Kalman filter to predict the motion trajectories and bounding boxes of the target objects in subsequent frames; second, calculating the IoU between the predicted trajectory bounding boxes and the detected bounding boxes, and leveraging re-identification technology (ReID) to associate the latest frame detection results with the current target trajectory set, ensuring the accurate generation of temporal information for multiple targets.

The specific process is illustrated in the Figure 4. Initially, detection boxes are categorized into high-confidence and low-confidence boxes based on their confidence scores. High-confidence boxes are then associated with previous tracking trajectories in the first matching step. In the second step, low-confidence boxes and the tracking trajectories that did not match with high-confidence boxes in the first attempt (e.g., objects that are heavily occluded in the current frame and thus have a decreased score) are matched. For detection boxes that do not match any tracking trajectory but have a sufficiently high score, a new tracking trajectory is initiated. As for the tracking trajectories that fail to match any detection boxes in both attempts, they are retained for 30 frames, and re-matching is performed when they reappear.



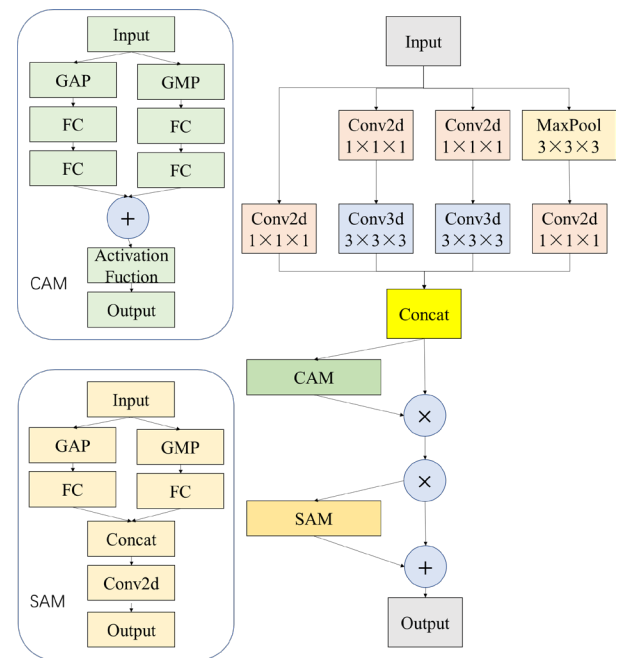
In the algorithm, the Kalman filter state vector is an octet, denoted as $[x_c, y_c, a, h, \dot{x}_c, \dot{y}_c, \dot{a}, \dot{h}]^T$, where the first two elements (x_c, y_c) represent the coordinates of the center point of the predicted bounding box, the third element a is the aspect ratio of the bounding box, the fourth element h is the height of the bounding box, and the last four elements are the velocity information of the preceding four elements. During the matching process, both the IoU distance matrix and the cosine distance matrix are integrated for matching purposes. The IoU distance reflects the degree of overlap between the target bounding boxes, while the cosine similarity measures the similarity between the target's appearance features [7]. By doing so, the algorithm can take into account both the motion information and the appearance features of the targets [8], achieving more accurate and robust multi-target matching. The algorithm is effective in tracking targets and providing stable temporal information under complex conditions such as target occlusion, scene changes, and camera motion.

Figure 5. The architecture of 3D CNN

The 3D CNN utilizes the temporal information of individuals to complete the final action recognition. Designed based on the

Figure 6. The architecture of 3D CEM

image classification model Inception-v1, the 3D CNN is constructed by stacking multiple convolutional layers, pooling layers, and 3D convolutional enhancement modules, as shown in the Figure 5. The 3D CNN was pre-trained on the Kinetics dataset, which improved the network’s performance and reduced the time required for network convergence.



The 3D convolutional enhancement module (3D CEM), as illustrated in Figure 6, involves extending some 2D convolutional kernels into the temporal dimension to form 3D convolutional kernels, enabling the network to extract more temporal features. After obtaining the input features from the previous layer, multiple convolutional operations of different sizes are performed in parallel, thereby more comprehensively capturing multi-scale information within the images and

enhancing the diversity and comprehensiveness of feature extraction. Subsequently, a feature enhancement structure is concatenated, which, through channel attention mechanisms (CAM) and spatial attention mechanisms (SAM), allows the model to extract features from the output vectors of the upper layer. This constructs interdependencies between the model's channels and spatial dimensions, improving the model's focus on and extraction capability of features, thereby enhancing its classification performance [9].

The objective of CAM is to learn the significance of each channel in order to enhance the useful channel information and suppress irrelevant channel information. This is achieved through the following steps: Initially, each input channel is divided into two branches, which undergo global average pooling (GAP) and global max pooling (GMP) respectively, yielding the average feature intensity $z_{c,avg}$ and the maximum feature intensity $z_{c,max}$ of a channel. The formulas are presented in Equations (6) and (7), where $x_{c,i,j}$ denotes the pixel value at the i -th row and j -th column of the c -th channel in the input feature map, and H and W represent the height and width of the feature map, respectively.

$$z_{c,avg} = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_{c,i,j} \quad (6)$$

$$z_{c,max} = \max(x_{c,i,j}) \quad (7)$$

Subsequently, the average and maximum values obtained are inputted into two fully connected layers (FC1 and FC2) to compute the channel attention weights. The outputs of these two fully connected layers are then summed, and a Sigmoid activation function is applied to constrain the values between 0 and 1, yielding the channel attention weights A_c . Finally, A_c is element-wise multiplied with the input feature map x . The Sigmoid activation function is denoted by σ , and the overall computation is formulated as follows:

$$f_{c1} = FC_1([z_{c,avg}, z_{c,max}]) \quad (8)$$

$$f_{c2} = FC_2(f_{c1}) \quad (9)$$

$$A_c = \sigma(f_{c1} + f_{c2}) \quad (10)$$

$$x_{channel} = A_c \odot x \quad (11)$$

The objective of SAM is to learn the significance of different spatial locations in order to enhance the useful location information and suppress irrelevant location information. This is achieved through the following steps: Initially, the input feature map undergoes GMP and GAP operations. Subsequently, the two feature maps obtained from GMP and GAP are concatenated, forming a new feature map M_{concat} that encompasses the maximum and average value information of each channel at different spatial locations. Thereafter, the concatenated feature map is passed through a 1×1 convolutional layer to reduce the number of channels to 1. This operation generates an intermediate representation for computing spatial attention weights. The sigmoid activation function is employed to produce the spatial attention weights A_s , which are then element-wise multiplied with the input feature map x . The overall computation is formulated in Equations (12) to (14).

$$M_{concat} = \text{concat}(z_{s,avg}, z_{s,max}) \quad (12)$$

$$A_s = \sigma(\text{Conv}_{1 \times 1}(M_{concat})) \quad (13)$$

$$x_{spatial} = A_s \odot x \quad (14)$$

By element-wise multiplying the outputs of CAM and SAM with the input data, an enhanced feature representation is generated. This feature representation possesses superior representational capabilities, which can improve the performance of CNN in classification tasks.

III. EXPERIMENTAL RESULTS AND ANALYSIS

A. Dataset

We have collected a diverse set of human action videos from various scenarios, totaling 2302 clips, as depicted in Figure 7. These videos are divided into training, validation, and testing sets in a ratio of 6:2:2, respectively, for model training, validation, and testing. Each video contains a single individual and showcases only one type of action, with a duration of 10 seconds.

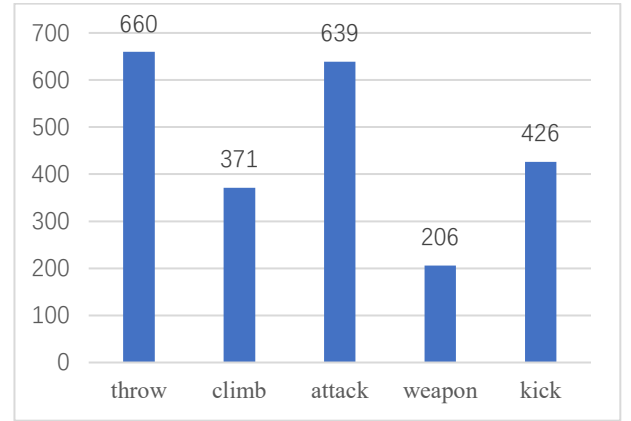


Figure 7. The statistical result of dataset

Utilizing the object detection model, we extract a frame containing the human target at fixed intervals, with the image size being slightly larger than the human target, thus forming a sequence of human action images, as illustrated in Figure 8.

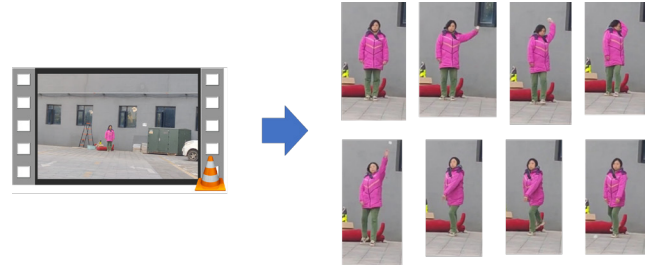


Figure 8. The sequence of human action

B. Model Training and Conversion

Model training was conducted using four NVIDIA Tesla V100 computation cards, with the deep learning framework PyTorch and the optimization algorithm being stochastic gradient descent with a learning rate of 0.01. The model was

trained for 80 epochs, and the loss function curve is depicted in Figure 9. It can be observed that as the number of training epochs increases, the loss values on both the training and validation sets decrease, thereby demonstrating the effectiveness of the model training.

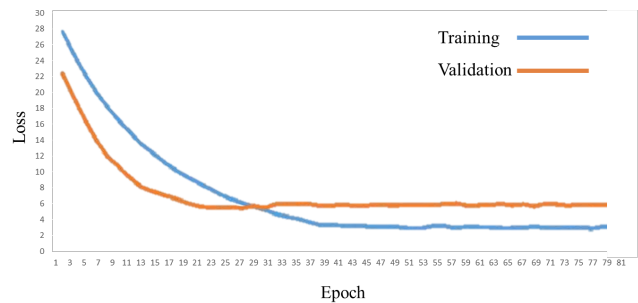


Figure 9. The sequence of human action

Initially, the PyTorch model obtained from training is converted into an ONNX model. Subsequently, the ascend tensor compiler (ATC) transformation tool is utilized to parse the ONNX model, followed by a series of processes including model topology preparation, model topology partitioning, model topology optimization, and model topology compilation. Throughout this process, ATC performs operations such as operator scheduling optimization, weight data rearrangement, and memory usage optimization, culminating in an offline model that is convenient for deployment on Atlas200DK. A comparison of the 2D CNN and 3D CNN models before and after conversion is presented in Table 1, demonstrating a reduction in model size by more than 50%, facilitating model deployment.

TABLE I. COMPARISON OF MODEL BEFORE AND AFTER CONVERSION

	2D CNN	3D CNN
Before conversion(.onnx)	42.6MB	86.2MB
After conversion(.om)	22.3MB	44.8MB

C. Model Inference

The experiments were conducted on the Atlas200DK, a product centered around the Ascend 310 AI processor, with a computational power of 11 TFLOPS in half-precision FP16, 8GB of memory, and a power consumption of 20W [10]. The average recognition frame rate was recorded at 10 fps. Evaluation metrics used were Precision and Recall. The recognition precision for each category is summarized in Table 2. It can be observed that the recognition precision for actions with more distinct features, such as climbing and carrying weapon, is relatively high. The recognition effect for attacking is moderate, while the recognition for throwing and kicking actions is comparatively poor. This is attributed to the fact that throwing and kicking are instantaneous actions, which are challenging to capture and recognize. The low Recall value for attacking is due to the significant flexible deformation of individuals when performing such actions, making it difficult to extract features from the surrounding environment.

TABLE II. EVALUATION RESULTS OF DATASET

	Precision/%	Recall/%
Throwing	55.8	88.6
Climbing	84.3	86.4
Attacking	75.2	65.3
Carrying weapons	88.3	89.2
Kicking	56.4	88.3

The locations of individuals are identified using rectangular bounding boxes, with the categories of behaviors annotated on the boxes, achieving the visualization of action recognition results, as depicted in Figure 10.

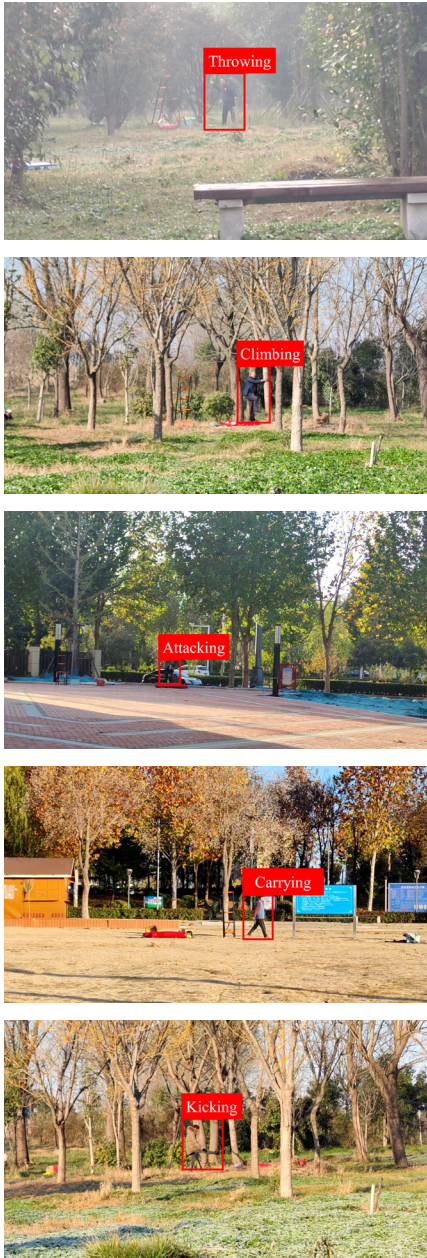


Figure 10. The visualization results of action recognition

IV. CONCLUSIONS

This paper addresses the challenges in human action extraction and modeling by designing a dual-network human action recognition method. Initially, a 2D CNN is designed to extract human targets from single-frame video images. Subsequently, a human matching module is utilized to generate temporal information for the human targets. Finally, a 3D CNN is implemented to recognize human actions. The method is deployed on the edge intelligence platform Atlas200DK, enabling online recognition of multiple human targets in video images.

REFERENCES

- [1] Wu F, Wang Q, Bian J, et al. (2023) A survey on video action recognition in sports: datasets, methods and applications. *IEEE Transactions on Multimedia*, 25: 7943-7966. <https://doi.org/10.1109/TMM.2022.3232034>.
- [2] Albawi S, Mohammed T A, Al-Zawi S. (2017) Understanding of a convolutional neural network. In: *International Conference on Engineering and Technology*, Antalya. pp. 1-6. <https://doi.org/10.1109/ICEngTechnol.2017.8308186>.
- [3] Koniusz P, L. Wang L, Cherian A. (2022) Tensor representations for action recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(2): 648-665. <https://doi.org/10.1109/TPAMI.2021.3107160>.
- [4] Carreira J, Zisserman A. (2017) Quo Vadis, action recognition? A new model and the kinetics dataset. In: *IEEE Conference on Computer Vision and Pattern Recognition*, Hawaii. pp. 4724-4733. <https://doi.org/10.1109/CVPR.2017.502>.
- [5] Tran D, Bourdev L, Fergus R, Torresani L, et al. (2015) Learning spatiotemporal features with 3d convolutional networks. In: *International Conference on Computer Vision*, Santiago. pp. 4489-4497. <https://doi.org/10.1109/ICCV.2015.510>.
- [6] Jiang P, Ergu D, Liu F, et al. (2022) A Review of Yolo algorithm developments. *Procedia Computer Science*, 199: 1066-1073. <https://doi.org/10.1016/j.procs.2022.01.135>.
- [7] Ye M, Shen J, Lin G, et al. (2021) Deep learning for person reidentification: A survey and outlook. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(6): 2872-2893. <https://doi.org/10.1109/TPAMI.2021.3054775>.
- [8] Ahmed E, Jones M, Marks T K. (2015) An improved deep learning architecture for person reidentification. In: *IEEE Conference on Computer Vision and Pattern Recognition*, Boston. pp. 3908-3916. <https://doi.org/10.1109/CVPR.2015.7299016>.
- [9] Brauwers G, Frasincar F. (2023) A general survey on attention mechanisms in deep learning. *IEEE Transactions on Knowledge and Data Engineering*, 35(4): 3279-3298. <https://doi.org/10.1109/TKDE.2021.3126456>.
- [10] Darawsheh A, Siam A, Shaar L, et al. (2022) High-performance detection and predication safety system using HUAWEI Atlas 200 DK AI developer kit. In: *International Conference on Computing and Information Technology*, Tabuk. pp. 213-216. <https://doi.org/10.1109/ICCIT52419.2022.9711553>.